OFFSIDE LABS

Meteora

DAMM v2 0.1.8

Smart Contract Security Assessment

January 2026

Prepared for:

Meteora

Prepared by:

Offside Labs

Sirius Xie

Contents

1 About Offside Labs

Offside Labs is a leading security research team, composed of top talented hackers from both academia and industry.

We possess a wide range of expertise in modern software systems, including, but not limited to, *browsers*, *operating systems*, *IoT devices*, and *hypervisors*. We are also at the forefront of innovative areas like *cryptocurrencies* and *blockchain technologies*. Among our notable accomplishments are remote jailbreaks of devices such as the **iPhone** and **PlayStation 4**, and addressing critical vulnerabilities in the **Tron Network**.

Our team actively engages with and contributes to the security community. Having won and also co-organized *DEFCON CTF*, the most famous CTF competition in the Web2 era, we also triumphed in the **Paradigm CTF 2023** within the Web3 space. In addition, our efforts in responsibly disclosing numerous vulnerabilities to leading tech companies, such as *Apple*, *Google*, and *Microsoft*, have protected digital assets valued at over **$300 million**.

In the transition towards Web3, Offside Labs has achieved remarkable success. We have earned over **$9 million** in bug bounties, and **three** of our innovative techniques were recognized among the **top 10 blockchain hacking techniques of 2022** by the Web3 security community.

 **https://offside.io/**

 **https://github.com/offsidelabs**

 **https://twitter.com/offside_labs**

2 Executive Summary

Introduction

Offside Labs completed a security audit of *DAMM v2* smart contracts, starting on January 23th, 2026, and concluding on January 27th, 2026.

Project Overview

DAMM v2 Release 0.1.8 introduces an internal position vesting mechanism to reduce reliance on extra vesting accounts and incorporates vesting state into the position state, enhances position splitting to cover vested portions and emits more complete on chain events, adds an on chain repair path to fix invalid base fee scheduler parameters that can block fee updates, and makes corresponding adjustments to base fee validation and related interfaces.

Audit Scope

The assessment scope contains mainly the smart contracts of the cp-amm program for the *DAMM v2* project.

The audit is based on the following specific branches and commit hashes of the codebase repositories:

- DAMM v2
  - Codebase: `https://github.com/MeteoraAg/damm-v2`
  - Release 0.1.8: `PR-177`
  - Commit Hash: df501080ff4ef537b8abbf1aa8a6a45dd1625906

The issues described in this report have been resolved by the following commit:

- DAMM v2
  - Codebase: `https://github.com/MeteoraAg/damm-v2`
  - Commit Hash: 96f13dea4c57ce49f6ab97bacdb4960b87d56771

We listed the files we have audited below:

- DAMM v2 Release 0.1.8
  - programs/cp-amm/src/base_fee/fee_market_cap_scheduler.rs
  - programs/cp-amm/src/base_fee/fee_rate_limiter.rs
  - programs/cp-amm/src/base_fee/fee_time_scheduler.rs
  - programs/cp-amm/src/base_fee/mod.rs
  - programs/cp-amm/src/constants.rs
  - programs/cp-amm/src/event.rs
  - programs/cp-amm/src/instructions/ix_lock_inner_position.rs
  - programs/cp-amm/src/instructions/ix_lock_position.rs
  - programs/cp-amm/src/instructions/ix_refresh_vesting.rs
  - programs/cp-amm/src/instructions/ix_remove_liquidity.rs
  - programs/cp-amm/src/instructions/ix_split_position.rs

- programs/cp-amm/src/instructions/ix_split_position2.rs
- programs/cp-amm/src/instructions/mod.rs
- programs/cp-amm/src/instructions/operator/ix_fix_config_fee_params.rs
- programs/cp-amm/src/instructions/operator/ix_fix_pool_fee_params.rs
- programs/cp-amm/src/instructions/operator/mod.rs
- programs/cp-amm/src/lib.rs
- programs/cp-amm/src/params/fee_parameters.rs
- programs/cp-amm/src/state/pool.rs
- programs/cp-amm/src/state/position.rs
- programs/cp-amm/src/state/vesting.rs

Findings

The security audit revealed:

- 0 critical issue
- 1 high issue
- 1 medium issue
- 1 low issue
- 1 informational issue

Further details, including the nature of these issues and recommendations for their remediation, are detailed in the subsequent sections of this report.

3 Summary of Findings

ID	Title	Severity	Status
01	Missing External Vesting Check for DestinationPosition in split_position2 IX	High	Fixed
02	Missing Static Check in fix_pool_fee_params IX	Medium	Fixed
03	Possible Unable to Lock InnerVesting When a Position Has External Vesting	Low	Fixed
04	Missing refresh_inner_vesting Operations	Informational	Fixed

4 Key Findings and Recommendations

4.1 Missing External Vesting Check for DestinationPosition in split_position2 IX

Severity: High	Status: Fixed
Target: Smart Contract	Category: Logic Error

Description

`first_position.inner_vesting` in the `split_position2` instruction, the new `position.vested_liquidity` is recalculated based on the updated `cliff_unlock_liquidity` and `liquidity_per_period` values after the split.

```
436         self.inner_vesting.cliff_unlock_liquidity =
                ↪ cliff_unlock_liquidity;
437         self.inner_vesting.liquidity_per_period = liquidity_per_period;
438
439         // recalculate total_released_liquidity
440         self.inner_vesting.total_released_liquidity = self
441             .inner_vesting
442             .get_max_unlocked_liquidity(current_point)?;
443
444         // recalculate vested liquidity
445         self.vested_liquidity =
                ↪ self.inner_vesting.calculate_remaining_vested_
446         liquidity()?;
```

programs/cp-amm/src/state/position.rs#L436-L445

However, before `second_position.vested_liquidity` is updated, only `second_position.inner_vesting.is_empty() == true` is required, and it is not checked whether `second_position` is still associated with any external vesting.

Impact

If a position with external vesting is used as `second_position` in `split_position2`, `second_position.vested_liquidity` will be updated to include a portion of `first_position.inner_vesting`. The liquidity associated with its external vesting will then no longer be reflected in `position.vested_liquidity`.

In subsequent `refresh_vesting` IXs, refreshing external vesting may trigger an underflow error due to insufficient `position.vested_liquidity`, preventing the external vesting from being refreshed again and permanently locking this portion of liquidity.

Recommendation

When `inner_vesting_liquidity_numerator > 0`, one of the following approaches can be applied:

- If `second_position` is allowed to have external vesting, add the `vested_liquidity` delta from the split inner vesting to `Position.vested_liquidity`.
- If `second_position` is not allowed to have external vesting, explicitly require `second_position.vested_liquidity == 0`.

Mitigation Review Log

Fixed in the commit 96f13dea4c57ce49f6ab97bacdb4960b87d56771.

4.2 Missing Static Check in fix_pool_fee_params IX

Severity: Medium	Status: Fixed
Target: Smart Contract	Category: Data Validation

Description

In `fix_pool_fee_params` IX, invalid base fee parameters are fixed into a validated state.

```
71      let fee_numerator_1 = base_fee_handler_1.get_min_fee_numerator()?;
72      let base_fee_mode_1 =
            ↪ pool.pool_fees.base_fee.base_fee_info.get_base_fee_
73      mode()?;
74
75      require!(
76          fee_numerator_0 == fee_numerator_1 && base_fee_mode_0 ==
        ↪ base_fee_mode_1,
77          PoolError::CannotUpdateBaseFee
78      );
79
80      // Ensure the new parameters are valid
81      base_fee_handler_1.validate(collect_fee_mode, activation_type)?;
```

programs/cp-amm/src/instructions/operator/ix_fix_pool_fee_params.rs#L71-L80

However, the current implementation only requires the old `base_fee_handler` to be static, and does not require the new `base_fee_handler` to be static at the `current_point`.

Impact

When `base_fee_mode` is a Time/MarketCap scheduler, checking only `min_fee_numerator` does not guarantee that the fee cannot be changed back to a higher value (e.g.,

`cliff_fee_numerator`).

When `base_fee_mode` is a Rate Limiter, if the old limiter has already expired, the new parameters can adjust values (e.g., increasing `max_limiter_duration`) to make the limiter become non-static again at the current point.

Recommendation

It's recommended to add a `validate_base_fee_is_static` check on `base_fee_handler_1` in the `fix_pool_fee_params` IX, to ensure the updated `base_fee_handler_1` is also static.

More conservatively, reference `fix_config_fee_params` and add a check on `max_fee_numerator` before and after the fix to ensure that `min/max/base_fee_mode` remain consistent before and after the fix.

Mitigation Review Log

Fixed in the commit 96f13dea4c57ce49f6ab97bacdb4960b87d56771.

4.3 Possible Unable to Lock InnerVesting When a Position Has External Vesting

Severity: Low	Status: Fixed
Target: Smart Contract	Category: Logic Error

Description

In `Position::refresh_inner_vesting` and `Position::apply_split_inner_vesting` , `inner_vesting` is only reset to `Default` when `inner_vesting.done()` and `vested_liquidity == 0` .

```
412         if self.inner_vesting.done()? && self.vested_liquidity == 0 {
413             self.inner_vesting = InnerVesting::default();
414         }
```

programs/cp-amm/src/state/position.rs#L412-L414

However, if a position is associated with external vesting, `position.vested_liquidity` may remain larger than zero even after `inner_vesting` has been fully released. In this case, `inner_vesting` will not be reset.

Impact

If `position.inner_vesting` is not reset due to the presence of external vesting, the user will not be able to lock new liquidity into `inner_vesting` via the `lock_inner_position`

IX.

Recommendation

If both inner vesting and external vesting are allowed to coexist, the reset condition for `position.inner_vesting` in `Position::refresh_inner_vesting` and `Position::apply_split_inner_vesting` should remove the `self.vested_liquidity == 0` check.

Mitigation Review Log

Fixed in the commit 96f13dea4c57ce49f6ab97bacdb4960b87d56771.

4.4 Informational and Undetermined Issues

Missing refresh_inner_vesting Operations

Severity: Informational	Status: Fixed
Target: Smart Contract	Category: Logic Error

In the `remove_liquidity`, `refresh_vesting`, and `split_position2` IXs, `Position::refresh_inner_vesting` is called first to update `Position.unlocked_liquidity`.

However, the `lock_position`, `lock_inner_position`, and `permanent_lock_position` IXs deduct liquidity from `Position.unlocked_liquidity` without first invoking `Position::refresh_inner_vesting`. As a result, when a `Position` has an expired but not-yet-refreshed `InnerVesting`, validation and accounting may be performed against a smaller `unlocked_liquidity`, and the instruction may fail with `PoolError::InsufficientLiquidity`.

It's recommended to call `Position::refresh_inner_vesting` (or at least call it when `inner_vesting` is non-empty) before deducting liquidity in all IXs that subtract liquidity from `Position.unlocked_liquidity`, to ensure the operation is based on the latest unlocked state.

5 Disclaimer

This audit report is provided for informational purposes only and is not intended to be used as investment advice. While we strive to thoroughly review and analyze the smart contracts in question, we must clarify that our services do not encompass an exhaustive security examination. Our audit aims to identify potential security vulnerabilities to the best of our ability, but it does not serve as a guarantee that the smart contracts are completely free from security risks.

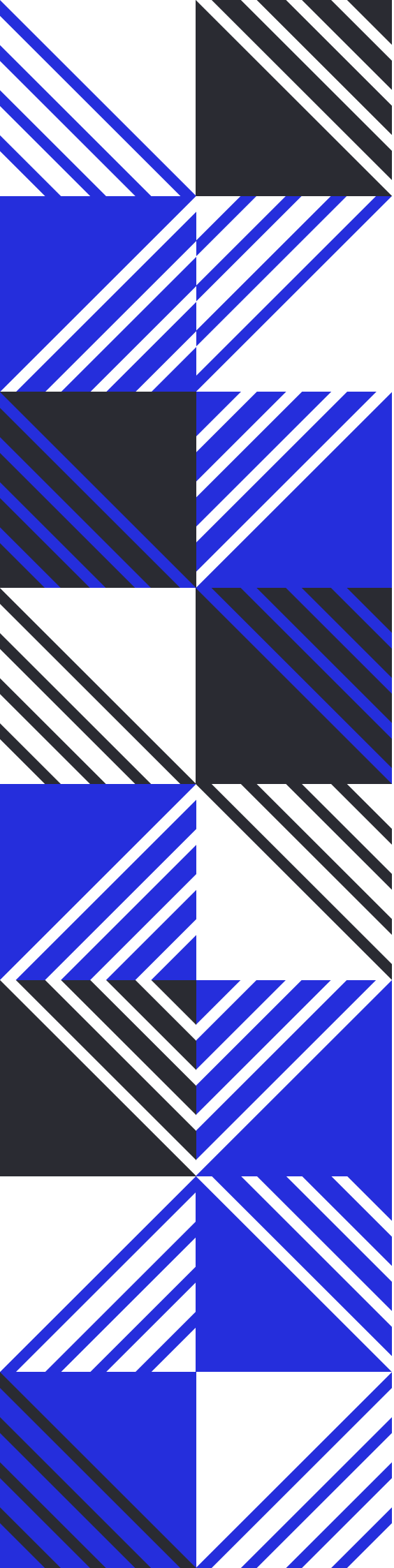
We expressly disclaim any liability for any losses or damages arising from the use of this report or from any security breaches that may occur in the future. We also recommend that our clients engage in multiple independent audits and establish a public bug bounty program as additional measures to bolster the security of their smart contracts.

It is important to note that the scope of our audit is limited to the areas outlined within our engagement and does not include every possible risk or vulnerability. Continuous security practices, including regular audits and monitoring, are essential for maintaining the security of smart contracts over time.

Please note: we are not liable for any security issues stemming from developer errors or misconfigurations at the time of contract deployment; we do not assume responsibility for any centralized governance risks within the project; we are not accountable for any impact on the project's security or availability due to significant damage to the underlying blockchain infrastructure.

By using this report, the client acknowledges the inherent limitations of the audit process and agrees that our firm shall not be held liable for any incidents that may occur subsequent to our engagement.

This report is considered null and void if the report (or any portion thereof) is altered in any manner.



OFFSIDE LABS

https://offside.io/

https://github.com/offsidelabs

https://twitter.com/offside_labs